Getter, Setter & Stack Overflow

Securing Object Properties and Avoiding Infinite Loops

1. Overview & Why This Matters

The Big Picture

When you build a class in JavaScript, you usually store data directly inside it (like `this.password = "123"`). The problem? Anyone using your class can read the raw password directly, or worse, change it to something invalid (like `this.password = ""`). We need a way to intercept someone trying to read or write a property.

Getters (`get`) and **Setters (`set`)** are special methods that allow you to hijack normal property access. They let you execute custom logic (like hiding the password or validating the input) exactly at the moment someone tries to read or modify your data.

Mental Model: The Security Guards

Imagine your Object is a highly secure Bank Vault.

- **Without Getters/Setters:** Anyone can walk into the bank, open the vault, and take or leave money directly. It's chaos.
- **With a Setter (The Deposit Guard):** When someone tries to put money in the vault (`user.password = "abc"`), they MUST hand it to the Deposit Guard first. The guard checks if the money is fake. If it's real, the guard puts it in the vault.
- **With a Getter (The Withdrawal Guard):** When someone tries to look at the money (`console.log(user.password)`), they ask the Withdrawal Guard. The guard might decide to hand them the money, or hand them an encrypted version of the money instead!

2. Deep-Dive Core Concepts

- `get` **keyword:** Placed before a method name. It turns that method into a "Getter." When you type `object.property`, it doesn't fetch a raw value; it secretly runs this function and returns the result.
- `set` **keyword:** Placed before a method name. It turns that method into a "Setter." When you type `object.property = value`, it secretly runs this function and passes your value as an argument.
- **The Dual Mandate:** In modern JavaScript, if you create a Getter for a property, you **must** also create a Setter for it. If you only create a Getter, the property becomes completely Read-Only, and trying to change it will just be ignored or throw an error.
- **The Backing Property (`_` underscore convention):** A private, hidden variable where the actual raw data is stored behind the scenes. Without this, your Getters and Setters will crash the program.

3. Step-by-Step Code Walkthroughs

Snippet 1: The Race Condition (Stack Overflow Bug)

This is the most common mistake beginners make. Let's see what happens when the constructor and the setter fight each other.

```
class User { constructor(email, password) { this.email = email; this.password = password; // 1. This triggers the Setter } get password() { return this.password; } // ❌ THE WRONG WAY set password(value) { // 2. Inside the Setter, we try to set 'this.password' again... // This triggers the Setter AGAIN! Infinite loop! this.password = value; } } const hitesh = new User("h@hitesh.ai", "abc"); // CRASH! RangeError: Maximum call stack size exceeded
```

Execution Trace (The Infinite Loop):
1. The constructor sets `this.password = "abc"`. Because we defined a `set password()`, JavaScript says, "Wait, don't save it! Send it to the Setter guard."
2. The Setter function takes "abc" and says, "Okay, save it to `this.password = "abc"`."
3. JavaScript intercepts this again! "Wait, don't save it! Send it to the Setter guard."
4. The Setter takes it, tries to save it, gets intercepted... over and over until the computer's memory (the Call Stack) overflows and the program violently crashes.

Snippet 2: The Fix using the Underscore (`_`)

To fix this, the Setter guard needs a *secret vault* to put the money in, so it stops triggering itself. We create a completely new variable by adding an underscore (`_password`).

```
class User { constructor(email, password) { this.email = email; this.password = password; } // The Getter guard fetches from the secret vault get password() { // Let's encrypt the password before returning it! return this._password.toUpperCase() + "hitesh"; } // ✅ THE RIGHT WAY set password(value) { // We save the data into a brand new variable: _password // This does NOT trigger the 'password' setter again. this._password = value; } } const hitesh = new User("h@hitesh.ai", "abc"); console.log(hitesh.password); // Output: ABChitesh
```

Snippet 3: The Old Way (Function-Based Legacy Code)

Modern ES6 Classes use the `get` and `set` keywords. But what if you are working on a 10-year-old codebase that uses Constructor Functions? You have to use the `Object.defineProperty()` tool to manually attach getters and setters.

```
function User(email, password) { // We establish the secret vaults immediately this._email = email; this._password = password; // We manually attach a Getter and Setter for 'email' Object.defineProperty(this, 'email', { get: function() { return this._email.toUpperCase(); }, set: function(value) { this._email = value; } }); } const chai = new User("chai@chai.com", "chai"); console.log(chai.email); // Output: CHAI@CHAI.COM
```

4. Common Beginner Pitfalls & "Gotchas"

⚠️ Calling a Getter like a Function

Even though a Getter is written exactly like a function (`get password() { ... }`), you **never** use parentheses when calling it. Treat it exactly like a normal property.

Wrong: `console.log(hitesh.password());` (This will crash with "password is not a function")
Right: `console.log(hitesh.password);`

> 💡 **The Constructor is still the boss!**
>
> When you set `this.password = password` inside your `constructor()`, you might wonder if it bypasses the Setter. It does **not** bypass it. When the constructor runs that line, it triggers the Setter immediately, passing your initial password straight into the `set` function!

5. Practical Cheat Sheet Summary

- `get`: Intercepts someone trying to **read** a property. Returns a value.
- `set`: Intercepts someone trying to **write** to a property. Must accept exactly one parameter (the new value).
- **The `_` Convention**: Used to name the *actual* variable where data is stored (e.g., `_password`) so the Getter and Setter don't accidentally call themselves infinitely.
- **No Parentheses**: Access getters normally (`obj.name`), and use setters with an equal sign (`obj.name = "John"`).

6. Comprehensive Knowledge Check

Test your understanding of Getters, Setters, and Memory Stacks with these 5 foundational questions.

Q1: What exact error occurs if you try to assign `this.password = value` directly inside the `set password()` method?

Answer: A "RangeError: Maximum call stack size exceeded". The setter tries to set the property, which triggers the setter again, creating an infinite loop that blows up the computer's call stack.

Q2: How does the `_` (underscore) convention solve the infinite loop problem?

Answer: By changing the internal variable name (e.g., to `_password`), the setter is no longer calling itself. It is successfully intercepting requests for `password`, but safely storing the data in a totally different box called `_password`.

Q3: If you only define a `get` method for a property, but you completely forget to define a `set` method, what happens if you try to change the property?

Answer: The property becomes Read-Only. If you try to assign a new value to it, JavaScript will either silently ignore the assignment, or (in Strict Mode) it will throw a TypeError.

Q4: If you have a legacy application using function-based objects, what built-in JavaScript method do you use to create getters and setters?

Answer: `Object.defineProperty(this, 'propertyName', { get: function(){}, set: function(){} })`.

Q5: Should you use parentheses when accessing a property that is powered by a `get` method?

Answer: No. Even though they are written as functions, you access them exactly like normal properties (e.g., `user.password`, not `user.password()`).